Level	Dirty read	Non-repeatable read	Phantom read
SERIALIZABLE	❌ prevented	❌ prevented	❌ prevented

Experiment 9 — Railway Booking (5,000 rows) and Index Benchmark

Aim

Create a Railway Booking table with at least 5,000 records. Benchmark query performance before and after creating single and composite indexes. Use EXPLAIN to compare execution plans.

Source Code

Setup and data generation

sql

```
-- ============================================================
-- Experiment 09 : Railway Booking indexes (5,000 rows)
-- MySQL 8.0
-- ============================================================
DROP DATABASE IF EXISTS RailwayDB;
CREATE DATABASE RailwayDB;
USE RailwayDB;


CREATE TABLE Railway_Booking (
    booking_id   INT AUTO_INCREMENT PRIMARY KEY,
    pnr          CHAR(10),
    passenger    VARCHAR(50),
    train_no     INT,
    train_name   VARCHAR(50),
    source_stn   VARCHAR(30),
```

```
    dest_stn     VARCHAR(30),
    travel_date  DATE,
    class        CHAR(2),
    fare         DECIMAL(8,2),
    booking_time DATETIME
);

-- Generate 5000 rows via recursive CTE
SET SESSION cte_max_recursion_depth = 10000;

INSERT INTO Railway_Booking
    (pnr, passenger, train_no, train_name,
     source_stn, dest_stn, travel_date, class, fare, booking_time)
WITH RECURSIVE seq AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM seq WHERE n < 5000
)
SELECT
    CONCAT('PNR', LPAD(n, 7, '0')),
    CONCAT('Passenger', n),
    12000 + (n % 20),
    CONCAT('Train', 12000 + (n % 20)),
    ELT(1 + (n % 5),       'Delhi','Mumbai','Chennai','Kolkata','Bengaluru'),
    ELT(1 + ((n + 2) % 5), 'Delhi','Mumbai','Chennai','Kolkata','Bengaluru'),
    DATE_ADD('2025-01-01', INTERVAL (n % 365) DAY),
    ELT(1 + (n % 4),       'SL','3A','2A','1A'),
    500 + (n % 2000),
    NOW() - INTERVAL (n % 1000) HOUR
FROM seq;
```

SELECT COUNT(*) FROM Railway_Booking;

Benchmark queries (BEFORE indexes)

sql

```
-- Q1: search by PNR
EXPLAIN SELECT * FROM Railway_Booking WHERE pnr = 'PNR0001000';

-- Q2: search by passenger name
EXPLAIN SELECT * FROM Railway_Booking WHERE passenger = 'Passenger2500';

-- Q3: search by train_no and travel_date
EXPLAIN SELECT * FROM Railway_Booking
WHERE train_no = 12005 AND travel_date = '2025-06-15';

-- Q4: source-dest with date range
EXPLAIN SELECT * FROM Railway_Booking
WHERE source_stn = 'Delhi' AND dest_stn = 'Chennai'
  AND travel_date BETWEEN '2025-03-01' AND '2025-03-31';
```

Create indexes

sql

```
CREATE UNIQUE INDEX idx_pnr          ON Railway_Booking(pnr);
CREATE INDEX        idx_passenger    ON Railway_Booking(passenger);
CREATE INDEX        idx_train_date   ON Railway_Booking(train_no, travel_date);
CREATE INDEX        idx_route_date   ON Railway_Booking(source_stn, dest_stn,
travel_date);
```

Benchmark queries (AFTER indexes)

sql

```
EXPLAIN SELECT * FROM Railway_Booking WHERE pnr = 'PNR0001000';
EXPLAIN SELECT * FROM Railway_Booking WHERE passenger = 'Passenger2500';
EXPLAIN SELECT * FROM Railway_Booking
WHERE train_no = 12005 AND travel_date = '2025-06-15';
```

```
EXPLAIN SELECT * FROM Railway_Booking
WHERE source_stn = 'Delhi' AND dest_stn = 'Chennai'
  AND travel_date BETWEEN '2025-03-01' AND '2025-03-31';
```

Timing comparison

```
sql
-- Before indexes (re-run by dropping indexes if you want a clean number)
SET @t1 = NOW(6);
SELECT COUNT(*) FROM Railway_Booking
WHERE source_stn = 'Delhi' AND dest_stn = 'Chennai'
  AND travel_date BETWEEN '2025-03-01' AND '2025-03-31';
SELECT TIMEDIFF(NOW(6), @t1) AS elapsed;
```

Output

Query	Plan before index	Plan after index
Q1 (PNR)	type=ALL, key=NULL, rows=5000, Extra=Using where	type=const, key=idx_pnr, rows=1, Extra=NULL
Q2 (passenger)	type=ALL, rows=5000	type=ref, key=idx_passenger, rows=1
Q3 (train_no + date)	type=ALL, rows=5000	type=ref, key=idx_train_date, rows≈15
Q4 (route + date)	type=ALL, rows=5000	type=range, key=idx_route_date, rows≈80

Row-count drop: 5000 → 1 or ~80 for all queries. Full-table scans eliminated.

Timing (typical dbfiddle MySQL 8):

Query	Before index	After index
Q1	~5 ms	<1 ms